# Workshop No. 3 - Backend Implementation and Testing

## Software Engineering Seminar


### Universidad Distrital Francisco José de Caldas

**Systems Engineering Program**

**Presented by:**

Brayan Alejandro Riveros Rodríguez
*20201020084*

Carlos Andrés Pescador Castro
*20182020139*

Cristian Santiago Ríos Vásquez
*20201020107*


Bogotá D.C., Colombia

November 8, 2025

# 1 Database Connection Scripts and Configuration

## 1.1 Overview

The authentication backend for the **Course Finder System** uses a MySQL 8 database for secure storage of administrator credentials. The connection between the Spring Boot backend and the database is managed via `Spring Data JPA` and configured in the `application.properties` file.

## 1.2 Database Initialization Script

The database schema is initialized through the SQL script located at `init_db_scripts/schema_db.sql`.

This script ensures that the database and required tables are properly created during setup.

Listing 1: Database initialization script.

```
-- Database initialization script for auth-back
-- Database: auth_db_course

CREATE DATABASE IF NOT EXISTS auth_db_course
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
USE auth_db_course;

DROP TABLE IF EXISTS admins;

CREATE TABLE admins (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
      ON UPDATE CURRENT_TIMESTAMP,
    INDEX idx_username (username),
    INDEX idx_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
  COLLATE=utf8mb4_unicode_ci;
```

## 1.3 Connection Configuration

The connection to MySQL is configured within the `application.properties` file as follows:

Listing 2: Spring Boot MySQL configuration.

```
spring.datasource.url=jdbc:mysql://localhost:3306/auth_db_course?
   useSSL=false&serverTimezone=UTC
```

```
spring.datasource.username=root
spring.datasource.password=YOUR_PASSWORD
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.
   MySQL8Dialect
server.port=8080
```

**Note:** Replace `YOUR_PASSWORD` with your local MySQL credentials.

### 1.4 Testing the Connection

Once configured, the connection can be tested by running:

```
mvn spring-boot:run
```

A successful connection will output:

```
Tomcat initialized with port 8080 (http)
Started AuthBackApplication in 3.2 seconds
```

## 2 REST API Documentation

### 2.1 Overview

The backend exposes a REST API that provides authentication features for the Course Finder System. All endpoints are fully documented using **Swagger UI**, which is automatically generated when the application starts.

### 2.2 Accessing Swagger UI

Once the application is running, the API documentation is available at:

`http://localhost:8080/swagger-ui/index.html`

Swagger UI provides:

- Interactive exploration of available endpoints.
- The ability to send test requests directly from the browser.
- Visualization of request and response formats, headers, and status codes.

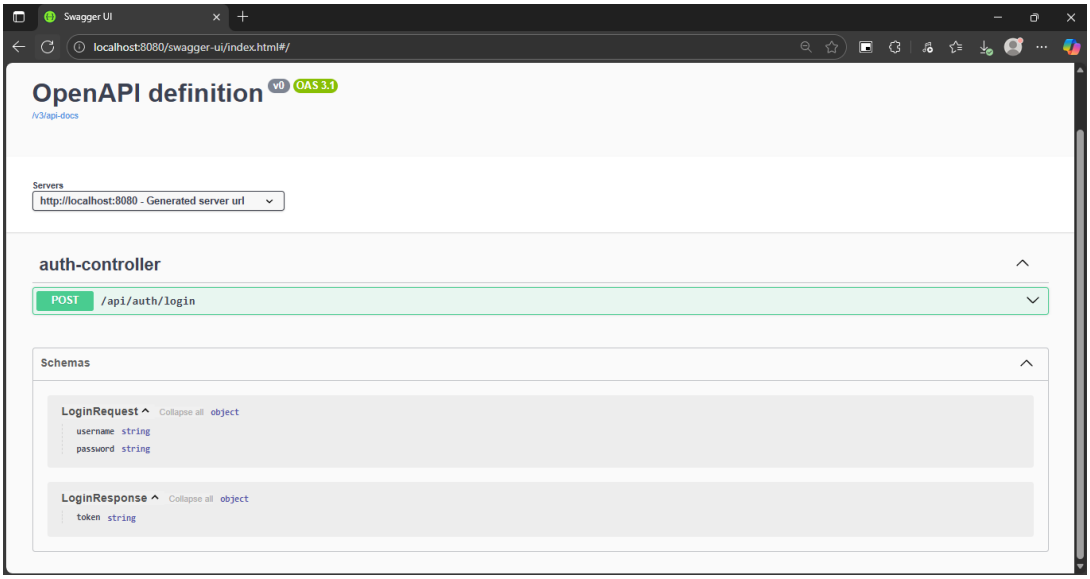


Figure 1: Swagger UI interface showing REST API endpoints.

## 2.3 Authentication Endpoint

| Method | Endpoint | Description |
|---|---|---|
| POST | /api/auth/login | Authenticates an admin and returns a JWT token. |

Table 1: Main REST API endpoint.

This endpoint handles administrator authentication within the system. It validates the provided username and password against the database and, if successful, generates a secure **JWT (JSON Web Token)** signed with the server's private key. The token can then be used in subsequent requests to access protected resources.

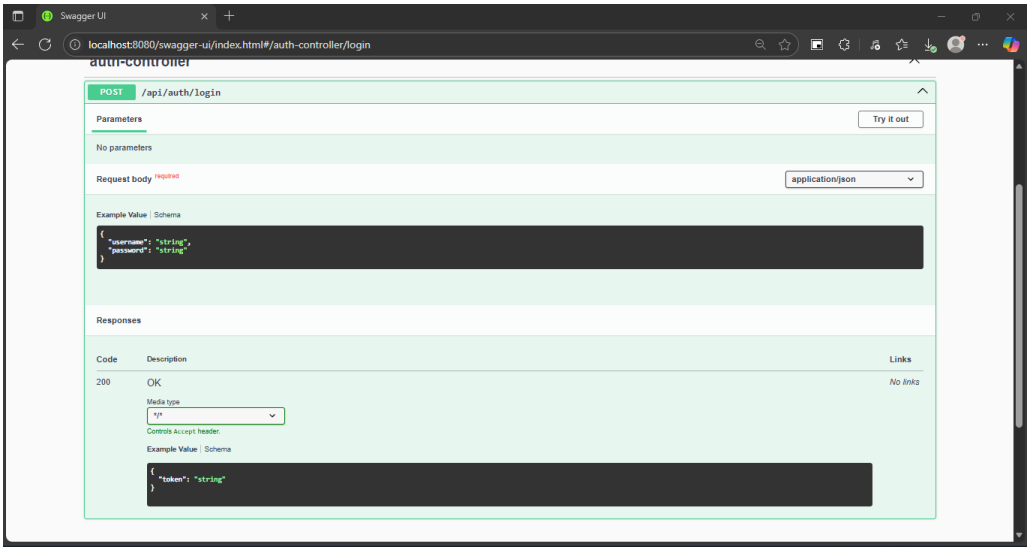


Figure 2: Authentication endpoint displayed in Swagger UI.

**Example request:**

```
{
  "username": "admin",
  "password": "admin123"
}
```

**Example response:**

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5..."
}
```

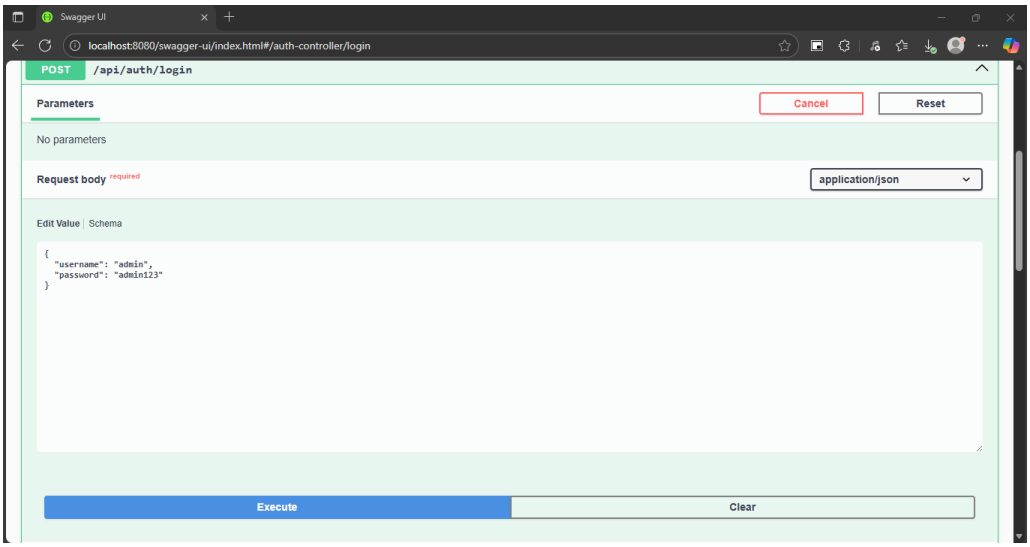


Figure 3: Example request executed from Swagger UI interface.

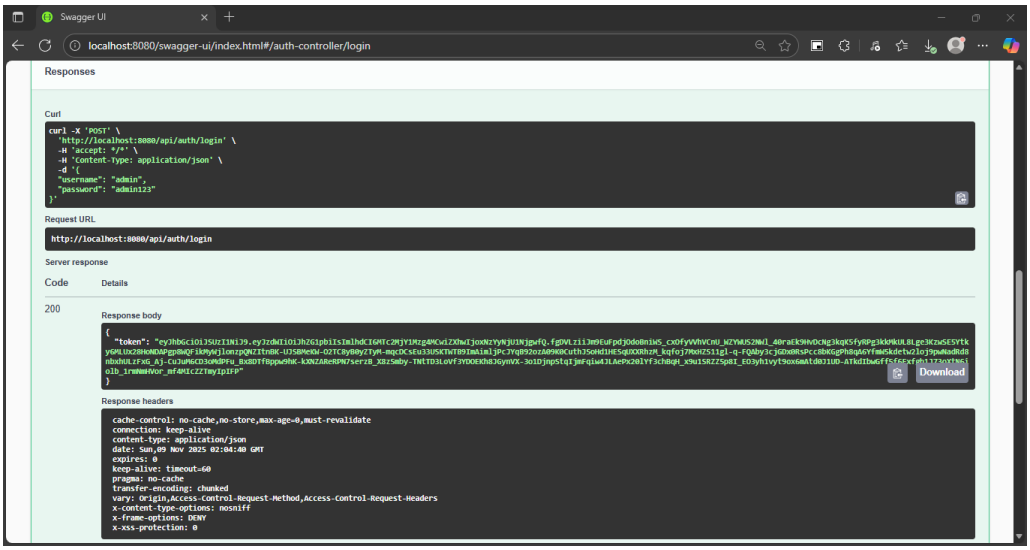


Figure 4: Example of a successful response returning a JWT token.

# 3 Unit Test Results and Code

## 3.1 Overview

Unit tests were implemented using **JUnit 5** and **Mockito** to validate the authentication logic and the generation of JWT tokens.

Tests are located in the `src/test/java/com/udistrital/authback/` directory.

## 3.2 AuthServiceTest

The `AuthServiceTest` class validates login logic, ensuring the authentication process behaves as expected.

- **Successful login:** Verifies that valid credentials return a JWT token.
- **Invalid password:** Ensures wrong credentials raise an exception.
- **User not found:** Confirms proper handling of missing user entries.

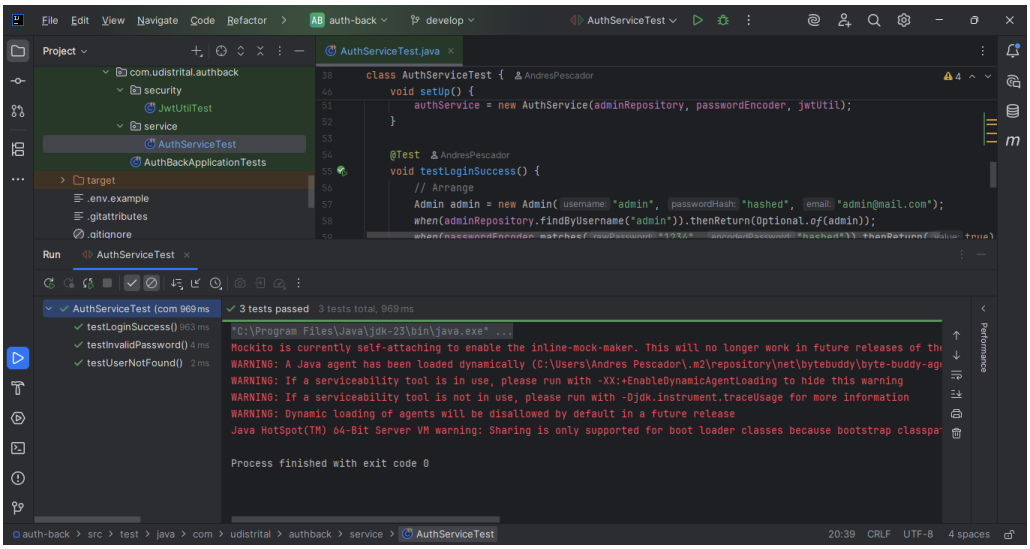


Figure 5: Successful execution of AuthService unit tests.

## 3.3 JwtUtilTest

The `JwtUtilTest` class tests the generation and signing of JWT tokens.

- **Token generation:** Validates that the token includes correct claims.
- **Signature verification:** Confirms the private key correctly signs tokens using RS256.
- **Expiration:** Checks that the token expiration time is applied correctly.

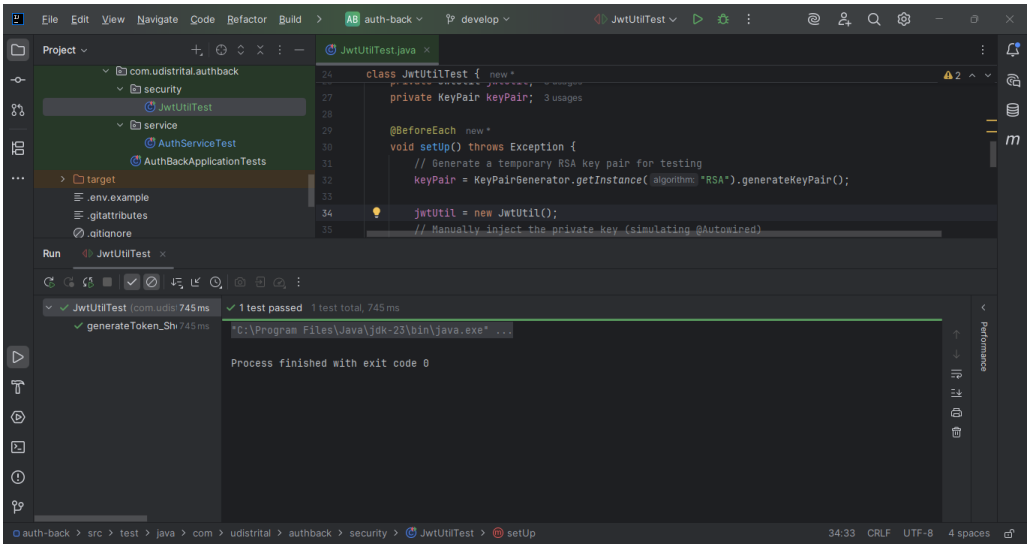


Figure 6: Successful execution of JwtUtil unit tests.

### 3.4 Command to Run Tests

All unit tests can be executed using the Maven command:

```
mvn test
```

Example console output

```
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

## 4 Conclusion

This backend successfully integrates a secure MySQL connection, JWT-based authentication, and verified functionality through comprehensive unit testing. Swagger UI ensures clear API documentation and facilitates testing for both developers and system administrators.